RMIT_Claude

The real-world problem

- Scam money moves through "mule" accounts that receive and pass on stolen funds.
- Malaysia's National Scam Response Centre handled 66,997 calls in 2023, and identified and disrupted 59,684 suspected mule accounts (per BNM's 2023 annual report, as reported in March 2024).
- **Bank Negara Malaysia already runs a National Fraud Portal**, launched in August 2024 with PayNet and the banks, for tracing funds and sharing fraud intelligence. Don't claim the problem is unaddressed.

Position yours as a complement: a demo of how this kind of sharing could be made operator-blind, policy-gated and member-governed. I couldn't confirm whether telcos and e-wallets take part in the portal, so don't claim a gap there either.

The constraint

Type	The tension
Privacy and law	Under the Financial Services Act 2013 (s.133), banks owe customers a duty of secrecy. The exceptions include enforcement requests and suspected offences, so sharing is allowed only in narrow cases. This comes from a 2016 article, so have a lawyer or your lecturer check it.
Custody	Whoever pools everyone's reports becomes the biggest breach target.
Authority	Whoever runs the pool can silently widen who sees what.
Trust	Competing institutions don't want to hand each other's data to a rival or a vendor.

How the application works

You'd build a working app for synthetic organisations: Bank A, Bank B, a telco, an e-wallet. **Never use real customer data.**

1. **Report:** A Bank A analyst files a mule report (account identifier, reason code, evidence). Details are encrypted in the browser.

2. **Check (tier 1):** A Bank B analyst looks up an account and sees a hit, a reason code and the reporting organisation. They don't see the evidence.
3. **Disclose (tier 2):** To see the evidence, Bank B files a request with a case reference. It needs approval from Bank A **and** an organisation that is neither of them. No one can approve their own request. Access expires after, say, 30 minutes.
4. **Govern:** Admitting a new member, or changing who can see what, needs a majority vote of member organisations.
5. **Audit:** One page lists every lookup, request, approval and vote.

Matching design (my own, to verify): Store account identifiers as keyed hashes. The hash key is itself encrypted so only member roles can decrypt it. Then a database dump reveals nothing useful. The weakness is that a member insider with the key could guess low-entropy account numbers, so state that limit openly and name private set intersection as the roadmap.

Where TideCloak is meaningful

Need	TideCloak feature
No shared password database to steal	BYOiD login for each analyst
Evidence and the hash key unreadable to the host	`doEncrypt` and `doDecrypt` with tags and role permissions
Tier-2 release needs approvers from distinct organisations, with expiry	Forseti policy (M-of-N approvers, no self-approval, time-locks)
No admin can quietly grant themselves access	Quorum-approved role and permission changes
Forged requests rejected	Server-side `verifyTideCloakToken` or the Next.js middleware

Swap test: with a normal identity provider and a database key, the operator holds the key, so the demo's "operator dumps the database" moment fails. That's why TideCloak is central.

Five-minute demo script

1. **Problem (30s):** the numbers above, plus "even with a national portal, one operator still sees everything."
2. **Report and check (60s):** Bank A files, Bank B gets a hit.

3. **Tier-2 request (60s):** Two teammates play the approvers. The self-approval attempt fails. Access then expires.
4. **Breach theatre (60s):** The "operator" dumps the database and gets ciphertext, then tries to grant themselves a role and is blocked.
5. **Governance (45s):** A live vote admits the e-wallet as a new member.
6. **Audit and close (30s):** Show the log and say what is simulated.

Build tiers and what to verify first

- **Must have:** BYOiD login, encrypted report, tier-1 lookup, and the operator-dump demo.
- **Should have:** the tier-2 approval flow and the audit page.
- **Could have:** the live vote.

Check these in the first hour:

1. **Cross-user decrypt:** Can a Bank B role decrypt data a Bank A user encrypted? The quickstart's `EncryptionRealmRole` and `DecryptionRealmRole` parameters suggest yes, but the SDK docs mention only "self" permissions.
2. **Quorum API:** I found no app-facing change-request API, so you may demo governance through the TideCloak admin console.
3. **User approval:** Whether approvers can be organisation officers in the policy setup.

Other risks: TideCloak was in beta in the sources I read, and the Fabric is a dependency, so record a backup video.

If you want, I can check the docs for items 1 to 3 and turn this into a task split for your team.

Sources:

- BNM: National Fraud Portal launch
- The Edge Malaysia: scam and mule account figures
- Mondaq: A Banker's Duty of Secrecy
- Forseti policy engine docs
- TideCloak Next.js reference
- Forseti crypto quickstart